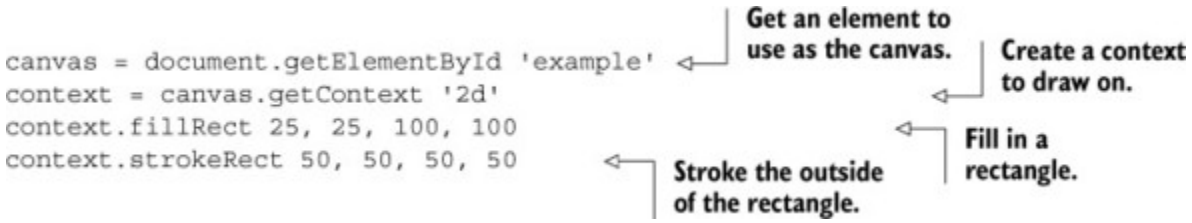
```
turtle = new Turtle
turtle.rotate(90).rotate(90)
```

If you're creating your own API, you can use this technique to make it fluent. You don't always write the API, though; oftentimes you only use it. For example, Scruffy didn't write the API for the turtle, but he has to use it. How can he use it with a fluent interface?

7.6.4. Chain

Not everything that can benefit from a fluent interface actually has one. Many of the APIs provided by web browsers are like this. Take the `canvas` API, for example. The `canvas` is covered in more detail in chapter 11, but for now you only need to know that it provides some drawing capabilities to web browsers, a little bit like the turtle:



See how you have to repeat the `context` every time, just like Scruffy had to repeat `turtle`? Instead of living with that, you can make your own fluent interface out of this nonfluent interface. Remember that `with` statement from JavaScript? You can declare something roughly similar in CoffeeScript using the `apply` method on a function:

```
using = (object, fn) -> fn.apply object

using turtle, ->
  @forward 2
  @rotate 90
  @forward 4
```

Close, but not quite the same. You (and Scruffy) really want a fluent interface that chains method calls: